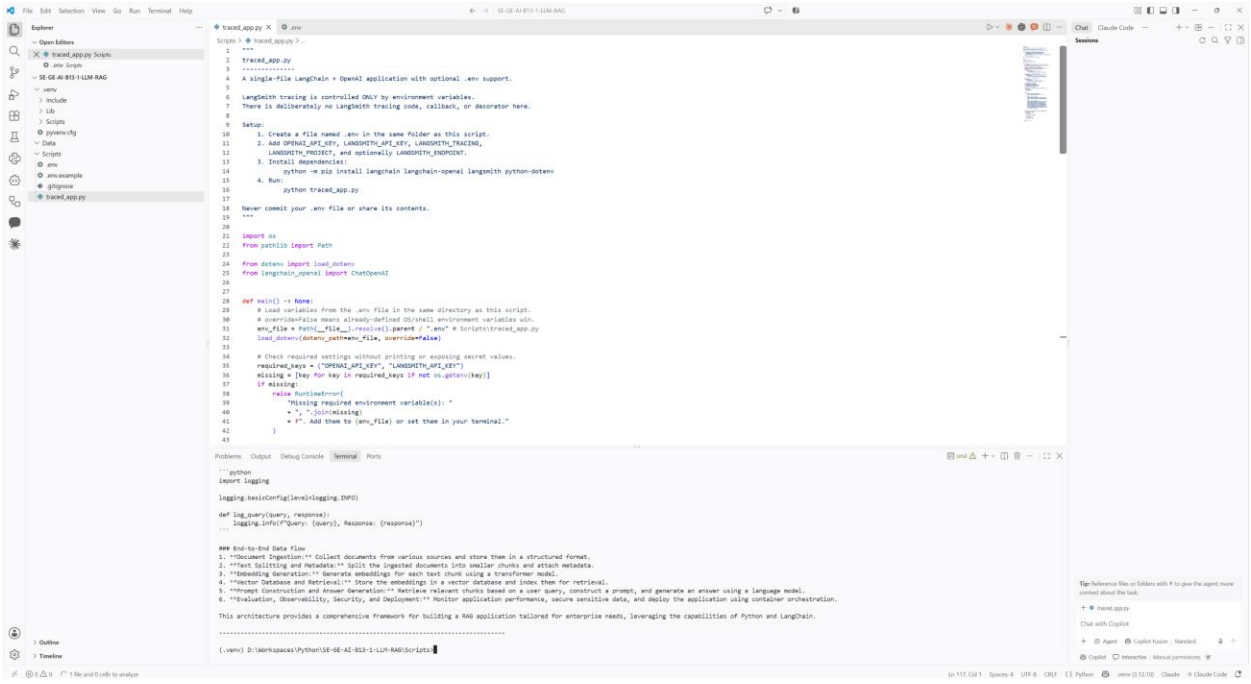


```python
"""
traced_app.py
--------------
A single-file LangChain + OpenAI application with optional .env support.

LangSmith tracing is controlled ONLY by environment variables.
There is deliberately no LangSmith tracing code, callback, or decorator here.

Setup:
    1. Create a file named .env in the same folder as this script.
    2. Add OPENAI_API_KEY, LANGSMITH_API_KEY, LANGSMITH_TRACING,
       LANGSMITH_PROJECT, and optionally LANGSMITH_ENDPOINT.
    3. Install dependencies:
           python -m pip install langchain langchain-openai langsmith python-dotenv
    4. Run:
           python traced_app.py

Never commit your .env file or share its contents.
"""

import os
from pathlib import Path

from dotenv import load_dotenv
from langchain_openai import ChatOpenAI


def main() -> None:
    # Load variables from the .env file in the same directory as this script.
    # override=False means already-defined OS/shell environment variables win.
    env_file = Path(__file__).resolve().parent / ".env" # Scripts\traced_app.py
    load_dotenv(dotenv_path=env_file, override=False)

    # Check required settings without printing or exposing secret values.
    required_keys = ("OPENAI_API_KEY", "LANGSMITH_API_KEY")
    missing = [key for key in required_keys if not os.getenv(key)]
    if missing:
        raise RuntimeError(
            "Missing required environment variable(s): "
            + ", ".join(missing)
            + f". Add them to {env_file} or set them in your terminal."
        )
```

```
```python
import logging

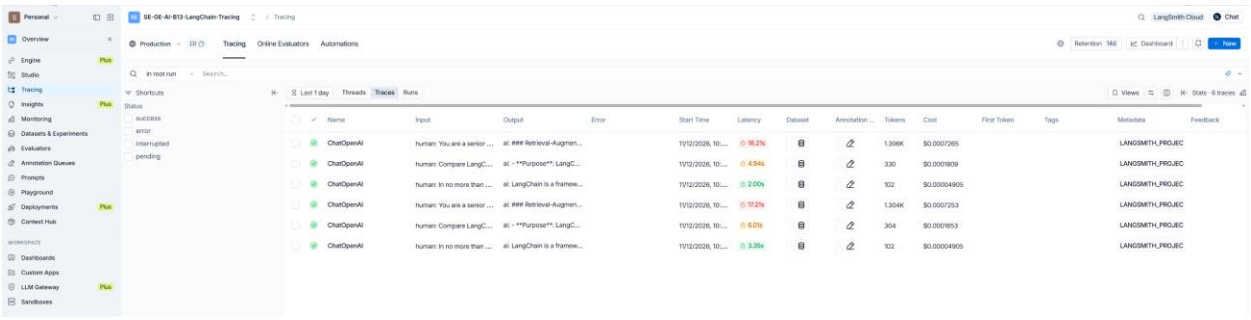
logging.basicConfig(level=logging.INFO)

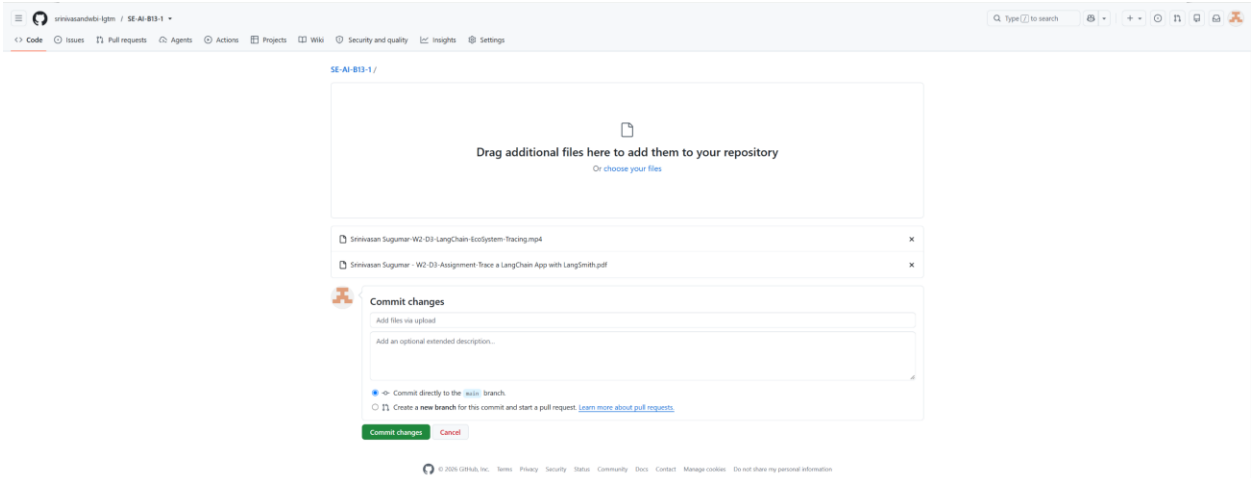
def log_query(query, response):
    logging.info(f"Query: {query}, Response: {response}")
```

End-to-End Data Flow
1. **Document Ingestion:** Collect documents from various sources and store them in a structured format.
2. **Text Splitting and Metadata:** Split the ingested documents into smaller chunks and attach metadata.
3. **Embedding Generation:** Generate embeddings for each text chunk using a transformer model.
4. **Vector Database and Retrieval:** Store the embeddings in a vector database and index them for retrieval.
5. **Prompt Construction and Answer Generation:** Retrieve relevant chunks based on a user query, construct a prompt, and generate an answer using a language model.
6. **Evaluation, Observability, Security, and Deployment:** Monitor application performance, secure sensitive data, and deploy the application using container orchestration.

This architecture provides a comprehensive framework for building a RAG application tailored for enterprise needs, leveraging the capabilities of Python and LangChain.

(.venv) D:\Workspaces\Python\SE-GE-AI-B13-1-LLM-RAG\Scripts>
```




srinivasandwbi-lgtm / SE-AI-B13-1
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
Type / to search
SE-AI-B13-1 /
Drag additional files here to add them to your repository
Or choose your files
Srinivasan Sugumar-W2-D3-LangChain-EcoSystem-Tracing.mp4
Srinivasan Sugumar - W2-D3-Assignment-Trace a LangChain App with LangSmith.pdf
Commit changes
Add files via upload
Add an optional extended description...
Commit directly to the main branch.
Create a new branch for this commit and start a pull request. Learn more about pull requests.
Commit changes
Cancel
© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information